

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE INGENIERÍA
PROGRAMACIÓN ORIENTADA A OBJETOS



Práctica #5
ArrayList

2105256
Luis Guillermo Maldonado Rodriguez

Felix Arredondo Santamaria
Martes 29 de Septiembre del 2026

Introducción:

En esta práctica se trabajará con la estructura de datos ArrayList en Java, aplicándola al desarrollo de un juego de cartas. Para ello, se implementarán las clases Card, Deck y Jugador, las cuales interactúan entre sí para representar las cartas, la baraja y los jugadores.

Durante la práctica se utilizarán métodos para agregar, eliminar, mostrar y modificar elementos de un ArrayList, además de implementar funciones para mezclar la baraja, obtener cartas, identificar la carta de mayor valor y comprobar si las cartas tienen la misma figura. También se utilizará el método toString() para facilitar la representación de los objetos.

Con esta práctica se busca reforzar el uso de colecciones de objetos, ciclos y métodos en programación orientada a objetos, así como comprender cómo diferentes clases pueden trabajar en conjunto para resolver un problema.

Competencias:

Desarrollar un programa utilizando distintas clases que interactúan, haciendo uso de colecciones de objetos y ciclos, con disposición para analizar un problema que se resuelve mediante la programación orientada a objetos.

Fundamentos:

¿Qué es la programación orientada a objetos?

Un modelo o un estilo de programación que nos da unas guías sobre cómo trabajar con él. Se basa en el concepto de clases y objetos. Este tipo de programación se utiliza para estructurar un programa de software en piezas simples y reutilizables de planos de código (clases) para crear instancias individuales de objetos.

¿Qué son los objetos?

Una clase es una plantilla. Define de manera genérica cómo van a ser los objetos de un determinado tipo. Por ejemplo, una clase para representar a animales puede llamarse ‘animal’ y tener una serie de atributos, como ‘nombre’ o ‘edad’ (que normalmente son propiedades), y una serie con los comportamientos que estos pueden tener, como caminar o comer, y que a su vez se implementan como métodos de la clase (funciones).

¿Qué es una clase?

Los objetos en programación representan cosas del mundo real, así como conceptos abstractos con sus características y comportamientos específicos. Un objeto cuenta con su estructura interna que combina variables, funciones y estructuras de datos. Usando el nombre del objeto y la sintaxis según el lenguaje de programación, puedes visualizar los valores del objeto y llamar las funciones que tiene predefinidas.

¿Qué es IntelliJ?

IntelliJ IDEA es un IDE creado por JetBrains diseñado especialmente para programadores que trabajan con Java, Kotlin y otros lenguajes de programación similares.

Procedimiento:

La Figura 1 muestra el diagrama de clases para un juego de cartas que se implementa en esta práctica. A continuación se describen los atributos y métodos que tienen las clases Card, Deck y Jugador.

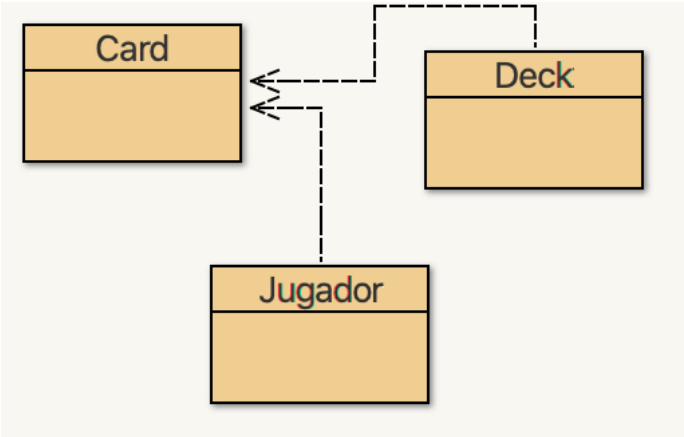


Figura 1. Diagrama de clases para un juego de cartas.

Card

Esta clase modela una carta de baraja inglesa. Tiene como atributos un valor y una figura. La clase debe tener un método String toString() que regresa una cadena con la representación de la carta como cadena. Así mismo, la clase tiene métodos getters y setters para sus atributos.

Deck

Esta clase modela una baraja de cartas. Tiene 52 cartas inicialmente. Pero, se pueden duplicar, triplicar, cuadruplicar, quintuplicar, sextuplicar, septuplicar, decuplicar o centuplicar. También tiene un método que muestra en la consola todas las cartas que contiene. Así mismo, tiene un método que revuelve las cartas. Tiene un método que regresa una carta que quita de la baraja. Tiene otro método que regresa una ArrayList con **N** cartas que quita de la baraja, donde N se recibe como parámetro. Se recomienda implementar un método String toString().

Jugador

Esta clase modela un jugador de cartas. Tiene un único atributo que es un ArrayList con cartas. Tiene un método para recibir un ArrayList con cartas. También tiene un método para mostrar en consola todas las cartas que tiene. Tiene un método que muestra y regresa la carta más alta que tiene. También un método que indique si las cartas tienen la misma figura. Se recomienda implementar un método String toString().

Desarrollo y Resultado de la práctica:

Código de la práctica:

Clase Deck:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Scanner;

public class Deck {
    private ArrayList<Card> deck;
    private Card carta;
    private Menu menu;
    private Scanner scn;

    public Deck(){
        deck = new ArrayList<Card>();
        menu = new Menu();
        scn = new Scanner(System.in);
    }

    public Card generarCarta(int valor, String simbolo){
        carta = new Card(valor, simbolo);
        deck.add(carta);
        return carta;
    }

    public void crearMazos() {
        int cantMazos = menu.preguntarMazos();
        int cantidadMazo;
        switch(cantMazos){
            case 8:
                cantidadMazo = 10;
```

```
            break;
        case 9:
            cantidadMazo = 100;
            break;
        default:
            cantidadMazo = cantMazos;

    }

    for (int o = 1; o <= cantidadMazo; o++){
        for (int i = 1; i <= 4; i++) {
            for (int c = 1; c <= 13; c++) {
                if(i == 1){
                    generarCarta(c,"♥");
                }else if(i == 2) {
                    generarCarta(c,"♣");
                }else if(i == 3) {
                    generarCarta(c,"♠");
                }else if(i == 4) {
                    generarCarta(c,"♦");
                }
            }
        }
    }

    Collections.shuffle(deck);
}

public void imprimirCartas(){
    int i = 1;
    for(Card card : deck)
    {
        System.out.println(card.cartaCompleta());
```

```
            if(i % 13 == 0){
                System.out.println("\n\n");
                if(i % 52 == 0){
                    System.out.println("\n\n");
                }
            }
            i++;
        }
        System.out.println("Se imprimieron " + deck.size() + " cartas.");

    }

    public Card darCarta(){
        Card cartaDar = deck.getFirst();
        deck.removeFirst();
        return cartaDar;
    }

    public ArrayList<Card> recogerVariasCartas(int cuantasCartasQuiere){

        ArrayList<Card> variasCartas = new ArrayList<>();
        for(int i = 0; i < cuantasCartasQuiere; i++){
            variasCartas.add(darCarta());
        }
        return variasCartas;
    }
}
```

Clase Card:

```
public class Card {
    private int valor;
    private String simbolo;
```

```
public Card(int valor, String simbolo){
    this.valor = valor;
    this.simbolo = simbolo;
}

public String cartaCompleta(){
     String val;
     switch (this.valor){
        case 1:
            val = "A";
            break;
        case 11:
            val = "J";
            break;
        case 12:
            val = "Q";
            break;
        case 13:
            val = "K";
            break;
        default:
            val = Integer.toString(this.valor);
            break;
    }
    return val + simbolo;
}

public int getValor(){
    return valor;
}

public String getSimbolo(){
```

```
        return simbolo;
    }
}
```

Clase Menu:

```
import java.util.Scanner;

public class Menu {
    private int opc;
    private Scanner scn;

    public Menu(){
        scn = new Scanner(System.in);
    }

    public int preguntarMazos(){
        System.out.println("Con cuantos mazos desea jugar?");
        System.out.println("[1] 1 Mazo.");
        System.out.println("[2] 2 Mazos.");
        System.out.println("[3] 3 Mazos.");
        System.out.println("[4] 4 Mazos.");
        System.out.println("[5] 5 Mazos.");
        System.out.println("[6] 6 Mazos.");
        System.out.println("[7] 7 Mazos.");
        System.out.println("[8] 10 Mazos.");
        System.out.println("[9] 100 Mazos.");
        if(scn.hasNext()){
            this.opc = scn.nextInt();
        }
        return opc;
    }
}
```

Clase Player:

```
import java.util.ArrayList;
```

```
public class Player {
  private ArrayList<Card> cartasMano;

  public Player(){
    cartasMano = new ArrayList<Card>();
  }

  public void tomarCarta(Deck mazoComunitario){
    cartasMano.add(mazoComunitario.darCarta());
  }

  public void imprimirCartasMano (){
    for (Card carta : cartasMano){
      System.out.println(carta.cartaCompleta());
    }
  }

  public Card determinarCartaMasAlta(){
    if(cartasMano.isEmpty()){
      return null;
    }
    Card cartaMasAlta = cartasMano.getFirst();

    for(Card carta : cartasMano){
      if(carta.getValor() > cartaMasAlta.getValor()){
        cartaMasAlta = carta;
      }
    }
    System.out.println(cartaMasAlta.cartaCompleta());
    return cartaMasAlta;
  }

  public boolean mismoPalo(){
```

```
        if(cartasMano.isEmpty()){
            return false;
        }

        String paloCarta = cartasMano.getFirst().getSimbolo();

        for (Card carta : cartasMano){
            if(!paloCarta.equalsIgnoreCase(carta.getSimbolo())){
                return false;
            }
        }
        return true;
    }
}
```

Clase Main:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main{
    public static void main(String[] args){

        System.out.println("\n===========Black Jack===========");

        Deck deck = new Deck();
        Deck variasCartas = new Deck();
        Player jugador = new Player();
        Scanner scn = new Scanner(System.in);

        deck.crearMazos();
        deck.imprimirCartas();
```

```
        System.out.println("\n\nToma 5 cartas:");

        for(int i = 0; i < 5; i++){
            jugador.tomarCarta(deck);
        }
        jugador.imprimirCartasMano();
        System.out.println("\n\n");

        System.out.println("Cartas sobrantes:\n");
        deck.imprimirCartas();

        System.out.println("Cuantas cartas quieres recoger del mazo de cartas?\n");
        int cuantasCartasQuiere = scn.nextInt();
        ArrayList<Card> cartas = deck.recogerVariasCartas(cuantasCartasQuiere);
        for (Card carta : cartas){
            System.out.println(carta.cartaCompleta());
        }

        System.out.println("\nCarta mas alta: ");
        jugador.determinarCartaMasAlta();
                        System.out.println("\nTienes cartas del mismo palo?: " +
jugador.mismoPalo());

        System.out.println("Cartas sobrantes:\n");
        deck.imprimirCartas();
    }
}
```

Imágenes de la ejecución:

```
===========Black Jack===========
Con cuantos mazos desea jugar?
[1] 1 Mazo.
[2] 2 Mazos.
[3] 3 Mazos.
[4] 4 Mazos.
[5] 5 Mazos.
[6] 6 Mazos.
[7] 7 Mazos.
[8] 10 Mazos.
[9] 100 Mazos.
4
4♥
K♠
9♦
A♥
8♠
A♥
9♠
5♠
2♠
A♠
4♠
7♥
Q♥
```

```
Se imprimieron 208 cartas.


Toma 5 cartas:
4♥
K♠
9♦
A♥
8♠
```

```
Se imprimieron 203 cartas.
Cuantas cartas quieres recoger del mazo de cartas?

7
A♥
9♣
5♣
2♠
A♠
4♣
7♥

Carta mas alta:
K♠

Tienes cartas del mismo palo?: false
```

```
Se imprimieron 196 cartas.
```

Aquí vemos cómo imprimimos 4 mazos de 52 cartas cada uno, por lo que tendremos 208 cartas impresas, de esas tomaremos 5 solamente para poder ejecutar las demás funciones, por lo que cuando tomemos 5 tendremos 203 cartas en el mazo del banco, después tomaremos unas 7 cartas para ver cual es la más alta y ver si todas tienen el mismo palo, por lo que finalmente tendremos 196 cartas.

Conclusiones:

Esta práctica me ayudó a comprender mejor cómo funcionan los ArrayList y cómo utilizar las funciones que tienen incorporadas. También me facilitó el aprendizaje conocer previamente el funcionamiento de los Arrays, ya que ambos tienen algunas características similares. Además, al ser una práctica base, me permitió comprender mejor estos conceptos y tener una mejor preparación para trabajar con estructuras de datos más adelante.

Referencias:

Martínez Canelo, M. (s. f.). *¿Qué es la programación orientada a objetos?* Profile. https://profile.es/blog/que-es-la-programacion-orientada-a-objetos/

EBAC. (2023, 21 de septiembre). *¿Qué es un objeto en programación?* https://ebac.mx/blog/objeto-en-programacion

Solano Gálvez, J. A. (2020). *Clases y objetos*. Unidades de Apoyo para el Aprendizaje, CUAED/Facultad de Ingeniería, UNAM. https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3068/mod_resource/content/1/UAPA-Clases-Objetos/index.html

Equipo de Contenidos de GoDaddy. (2024, 26 de agosto). *IntelliJ IDEA: Ventajas para el desarrollo*. GoDaddy. https://www.godaddy.com/resources/es/crearweb/intellij-idea-que-es